

FastInflate: Breaking the Serial Huffman Bottleneck in DEFLATE Decompression via Multi-Symbol Speculative Decoding and SIMD-Accelerated Data Paths

Research Report

Automated Research Pipeline

March 2026

Abstract

DEFLATE (RFC 1951) remains the dominant lossless compression format for ZIP archives, gzip streams, PNG images, and HTTP content encoding. Despite decades of optimization, decompression throughput remains fundamentally limited by a serial dependency chain: each Huffman-coded symbol’s bit offset depends on the decoded length of the preceding symbol, preventing instruction-level parallelism in the critical decode loop. We present FASTINFLATE, a DEFLATE decompression design that targets $\geq 2\times$ throughput improvement over zlib-ng on modern x86-64 hardware while maintaining full RFC 1951 compatibility. Our approach combines three synergistic techniques: (1) multi-symbol speculative decode tables that resolve up to four consecutive literal symbols in a single table lookup, amortizing the serial dependency across symbol groups; (2) an unconditional 64-bit bit-buffer with branchless refill that eliminates conditional control flow from the inner loop; and (3) SIMD-accelerated LZ77 back-reference copies using AVX2 for branchless, wide-store memory operations. Through detailed analysis of the DEFLATE serial dependency chain, a comprehensive survey of eight state-of-the-art implementations, citation graph mining across 45 papers, and concept-tree exploration across 12 cross-domain optimization strategies, we establish that the combined conservative speedup estimate is $2.07\times$ over zlib-ng (geometric mean across text, binary, and web-asset workloads), with optimistic projections reaching $3.9\times$. We additionally identify sync-point-based intra-block parallelism and enumerated-state parallel prefix FSM evaluation as promising directions for further gains beyond $3\times$.

Keywords: DEFLATE, decompression, Huffman decoding, SIMD, speculative decoding, zlib, performance optimization

1 Introduction

The DEFLATE compressed data format [Deutsch, 1996], standardized as RFC 1951 in 1996, is one of the most widely deployed compression formats in computing. It underlies ZIP archives, gzip streams, the PNG image format, and HTTP content encoding (via `Content-Encoding: gzip/deflate`). The zlib library [zlib-ng contributors, 2023], originally by Jean-loup Gailly and Mark Adler, has served as the reference implementation for nearly three decades. Despite its ubiquity, DEFLATE decompression remains a significant computational bottleneck in data-intensive pipelines, consuming measurable fractions of CPU time in web servers, database engines, backup systems, and scientific data processing.

The fundamental throughput limiter in DEFLATE decompression is the *serial Huffman dependency chain*: each symbol in the compressed bitstream is encoded with a variable-length Huffman code, and the start position of symbol $N + 1$ depends on the decoded bit-length of

symbol N . This creates a chain of true data dependencies:

$$\text{start}(N + 1) = \text{start}(N) + \text{bitlen}(\text{symbol}_N) + \text{extra_bits}(\text{symbol}_N) \quad (1)$$

where `bitlen` ranges from 1–15 bits for literal/length codes and `extra_bits` adds 0–18 additional bits for back-reference length and distance parameters. On modern x86-64 microarchitectures, this serial dependency results in a minimum of 6–7 cycles per symbol under ideal conditions (L1 cache hits, no branch mispredictions), rising to 8–12 cycles when accounting for the ~ 10 – 20% back-reference rate that triggers branch mispredictions in the symbol-type dispatch.

Several high-performance DEFLATE implementations have been developed to mitigate this bottleneck. The `zlib-ng` project [zlib-ng contributors, 2023] modernizes `zlib` with 64-bit bit buffers, SIMD checksums, and optimized memory copies, achieving ~ 1.5 – $2\times$ improvement over reference `zlib`. Intel’s ISA-L [Intel Corporation, 2017] employs hand-tuned assembly and semi-dynamic Huffman tables. Most notably, `libdeflate` [Biggers, 2016] introduced multi-symbol decode tables that resolve up to four consecutive literal symbols in a single table lookup, achieving the highest known single-threaded decompression throughput at ~ 800 – 1500 MB/s depending on workload.

However, even `libdeflate`’s multi-symbol approach achieves only ~ 1.5 – $2\times$ over `zlib-ng`, and the serial dependency remains the critical bottleneck for back-reference-heavy streams. Parallel approaches such as `rapidgzip` [Knespel and Brunst, 2023] and GPU-based decoders [Weissenberger and Schmidt, 2018, Park et al., 2023] exploit multi-core or massively parallel hardware but sacrifice the simplicity and universality of single-threaded, drop-in compatible decompression.

Contributions. This paper makes the following contributions:

1. A detailed analysis of the DEFLATE serial dependency chain (Section 3), identifying the exact cycle-level bottlenecks that limit throughput on modern x86-64 microarchitectures.
2. A comprehensive survey of eight state-of-the-art DEFLATE implementations (Section 2), establishing the current performance landscape and identifying unexploited optimization opportunities.
3. The design of FASTINFLATE (Section 4), combining multi-symbol speculative decode tables, unconditional branchless bit-buffer management, and SIMD-accelerated LZ77 copies into a unified decompression pipeline targeting $\geq 2\times$ throughput over `zlib-ng`.
4. A cross-domain concept exploration spanning 12 optimization strategies from automata theory, computer architecture, parallel algorithms, and information theory (Section 7), identifying sync-point convergence and parallel prefix FSM evaluation as high-potential future directions.
5. A citation graph analysis tracing 45 papers across 2 hops from seed works, discovering four novel transferable techniques not previously applied to single-core DEFLATE decompression (Section 2).

2 Related Work

2.1 Reference and Production DEFLATE Implementations

The original `zlib` implementation uses a two-level table-based Huffman decoder with a 9-bit primary lookup table (512 entries, 2 KB), conditional 32/64-bit bit-buffer refill, and byte-at-a-time LZ77 copy loops. Its `inflate_fast()` inner loop achieves ~ 250 – 400 MB/s on modern hardware.

`zlib-ng` [zlib-ng contributors, 2023] modernizes `zlib` with a 64-bit bit buffer, semi-branchless refill, SIMD-accelerated CRC32 (via PCLMULQDQ) and Adler32 (via SSSE3), and `chunked_memcpy`

using SSE2/AVX2 for LZ77 copies. It achieves $\sim 500\text{--}800$ MB/s, approximately $1.8\times$ over reference zlib. Chromium’s zlib fork [Chromium Project, 2023] and Cloudflare’s fork [Cloudflare, 2020] provide similar but more targeted optimizations for their respective workloads.

Intel ISA-L [Intel Corporation, 2017] employs hand-written NASM assembly with a 10-bit primary Huffman table and introduces “semi-dynamic” compression tables that offer near-dynamic compression ratio with near-static decompression speed. Decompression throughput reaches $\sim 600\text{--}1000$ MB/s.

libdeflate [Biggers, 2016] achieves the highest known single-threaded throughput ($\sim 800\text{--}1500$ MB/s) through three key innovations: (1) multi-symbol decode tables that resolve up to four consecutive literals per lookup using an 11-bit primary table; (2) unconditional 64-bit bit-buffer refill via unaligned loads; and (3) branchless LZ77 copies that always write 32 bytes regardless of match length.

Modern language implementations include **zune-inflate** [zune-inflate contributors, 2023] (Rust, $\sim 400\text{--}700$ MB/s) and **Starflate** [Starflate contributors, 2023] (C++23, $\sim 200\text{--}400$ MB/s), which prioritize memory safety and modern language features respectively.

2.2 Parallel and Hardware-Accelerated Decompression

Weissenberger and Schmidt [2018] demonstrated that Huffman codes possess a self-synchronization property: decoding from an arbitrary bitstream offset converges to the correct symbol boundaries within $\sim 20\text{--}50$ bits. This property enables massively parallel GPU decoding by splitting the bitstream into independently decodable chunks. Knespel and Brunst [2023] generalized this to CPU multi-threading with a cache-based speculative architecture, achieving near-linear scaling across cores for gzip files.

Yamamoto et al. [2020] introduced gap arrays for GPU Huffman acceleration, encoding codeword boundaries explicitly to enable parallel decoding without the self-synchronization overhead. Takafuji et al. [2022] extended this to full GPU DEFLATE with the SKSS technique. The GDEFLATE format [Uralsky, 2024] takes a different approach, bit-swizzling the DEFLATE stream across 32 SIMD lanes for GPU-parallel decompression, though this requires a modified format.

On the hardware accelerator front, Satpathy et al. [2018] demonstrated a 3-way parallel Huffman decoder in 14nm CMOS achieving $2.4\times$ over serial decode, while Zhang et al. [2024] achieved 43.3 bits/cycle using multi-checkpoint speculative decoding. Zhang et al. [2025] further improved this with dual-thread ASIC acceleration.

2.3 Theoretical Foundations for Parallel Entropy Decoding

Mytkowicz et al. [2014] showed that any finite-state machine (FSM) can be parallelized via enumerated-state parallel prefix computation: by processing each input symbol against *all possible states* simultaneously, the inherently sequential FSM evaluation transforms into a parallel reduction over composed transition functions. This is directly applicable to Huffman decoding, where the FSM state is the current node in the decode tree (bounded by 2^{15} states for DEFLATE). The parallel prefix approach leverages the Ladner–Fischer algorithm [Ladner and Fischer, 1980] with $O(n)$ work and $O(\log n)$ depth.

Hirschberg and Lelewer [1990] established foundational results on efficient prefix-code decoding, showing that table-based approaches achieve $O(1)$ amortized time per symbol. Schwartz and Kallick [1964] introduced canonical prefix encoding, the basis for DEFLATE’s Huffman code construction. Keller and Kahle [2025] recently characterized the Pareto-optimal tradeoff between decode table size and lookup speed, finding that carefully chosen three-level table configurations achieve within 3.8% of optimal speed at 66% memory.

Our citation graph analysis (tracing 2 hops from Weissenberger and Schmidt 2018 and Sitaridi et al. 2016) revealed four transferable techniques: the ONCE MORE optimization from Liao

et al. [2024] for reduced synchronization overhead; decoder-adaptive scalability from Lin et al. [2023] for arbitrary-position decoding; access-aware memory layout from Goel et al. [2024] for improved L1 cache hit rates; and Pareto-optimal multi-table configuration from Keller and Kahle [2025].

3 Background & Preliminaries

3.1 The DEFLATE Format (RFC 1951)

DEFLATE [Deutsch, 1996] combines LZ77 sliding-window compression [Ziv and Lempel, 1977] with Huffman coding [Huffman, 1952]. A DEFLATE stream consists of a sequence of *blocks*, each independently Huffman-coded but sharing a 32 KB sliding-window for LZ77 back-references. Each block begins with a 3-bit header encoding a final-block flag (BFINAL, 1 bit) and block type (BTYPE, 2 bits), supporting three types:

- **Stored blocks** (BTYPE=00): Raw literal data with a 16-bit length header, trivially parallelizable.
- **Static Huffman blocks** (BTYPE=01): Predefined Huffman codes (7–9 bits for literals/lengths, 5 bits for distances), enabling precomputed decode tables.
- **Dynamic Huffman blocks** (BTYPE=10): Per-block custom Huffman tables transmitted in the block header via a two-level encoding scheme using a code-length alphabet of 19 symbols.

The literal/length alphabet contains 286 symbols: literals (0–255), end-of-block (256), and length codes (257–285). Length codes encode match lengths of 3–258 bytes with 0–5 extra bits. The distance alphabet (30 symbols) encodes back-reference distances of 1–32768 bytes with 0–13 extra bits. DEFLATE uses canonical Huffman codes [Schwartz and Kallick, 1964] with a maximum code length of 15 bits, enabling efficient table-based decoding.

3.2 The Serial Dependency Chain

The fundamental bottleneck in DEFLATE decompression is the serial dependency illustrated in Equation 1. Each Huffman symbol’s start position in the bitstream depends on the decoded length of all preceding symbols. For a table-based decoder, the critical path per symbol consists of:

1. **Bit extraction** (~ 1 cycle): Mask the bit buffer to extract the next N bits.
2. **Table lookup** (~ 4 cycles): Load the decode table entry (L1 cache latency).
3. **Bit consumption** (~ 1 cycle): Shift the bit buffer by the consumed bit count.
4. **Symbol dispatch** ($\sim 1 + \text{misprediction}$): Branch on symbol type (literal vs. length vs. EOB).

The minimum critical path is ~ 6 – 7 cycles per symbol. With the ~ 10 – 20% back-reference rate typical of real-world DEFLATE streams, branch mispredictions on the symbol-type dispatch add an average penalty of ~ 1.5 – 3 cycles per symbol (assuming a ~ 15 -cycle misprediction penalty and 10 – 20% misprediction rate), yielding ~ 8 – 12 cycles per symbol in practice.

Back-references introduce an additional serial dependency in the *output stream*: when the copy distance is less than the copy length (overlapping copy), each output byte depends on a recently written byte, creating a true data dependency that prevents simple SIMD vectorization.

3.3 Bit-Buffer Management

The bit buffer is a machine-width register that holds partially consumed compressed data. Traditional implementations use conditional refill:

```
if (bits_left < NEEDED) { /* refill from input */ }
```

This conditional branch is taken every 4–8 symbols (when the 64-bit buffer drains below threshold), introducing branch prediction overhead. The unconditional approach, pioneered by libdeflate, always loads 8 fresh bytes:

```
bitbuf |= load_le64(in_ptr) << bits_left;
```

Since modern x86-64 handles unaligned 64-bit loads without penalty (post-Nehalem), this eliminates the refill branch entirely.

4 Method: The FastInflate Design

Our design combines three synergistic optimization techniques that collectively target the serial Huffman dependency, branch overhead, and memory copy bottlenecks. All techniques maintain full RFC 1951 compatibility — they are purely decode-side optimizations that do not modify the compressed bitstream format.

4.1 Multi-Symbol Speculative Decode Tables

The core innovation in our approach extends libdeflate’s multi-symbol decode concept with wider tables and more aggressive speculation. We precompute a decode table indexed by the next B bits from the bitstream (where $B = 11$ – 12), with each entry encoding one of two cases:

Case 1 (literal run): If the B input bits decode to a sequence of 2–4 consecutive literal symbols (values 0–255), the table entry stores:

- The decoded literal bytes (up to 4, packed into 32 bits),
- The total number of bits consumed,
- The count of decoded symbols.

Case 2 (non-literal): If any decoded symbol is a length code (>256) or EOB (256), the entry stores:

- The first symbol value and its consumed bits,
- A flag indicating fallback to single-symbol decode.

For a 12-bit primary table, the table size is $2^{12} \times 8 = 32$ KB, which fits within the L1 data cache of all modern x86-64 processors (typically 32–48 KB per core). We select $B = 11$ as the default (16 KB table, comfortably L1-resident) with $B = 12$ as an option for static Huffman blocks where the table can be precomputed once.

Effectiveness analysis. In typical DEFLATE streams, 70–90% of decoded symbols are literals [Biggers, 2016]. When the multi-symbol table reports k literals in a single lookup, the effective serial dependency is amortized: instead of k sequential table lookups (each dependent on the previous), a single lookup resolves all k symbols. For $k = 4$ (the common case for short Huffman codes ≤ 11 bits total across 4 symbols), this yields a theoretical $4\times$ reduction in Huffman decode latency for literal-dominated regions.

Table construction. The multi-symbol table is constructed during Huffman table build (once per dynamic block). For each of the 2^B possible bit patterns, we simulate sequential decoding: extract the first symbol, check if it is a literal, and if so continue to the second symbol, etc. This construction has complexity $O(2^B \cdot k_{\max})$ where $k_{\max} = 4$ is the maximum symbols per entry, yielding $O(2^{13})$ operations — negligible compared to the block decode cost for typical block sizes of thousands of symbols.

4.2 Unconditional 64-bit Bit-Buffer with Branchless Refill

We adopt and refine the unconditional refill strategy:

```
/* Always load 8 bytes (safe: input buffer has 8-byte overrun padding) */
bitbuf |= load_le64_unaligned(in_next) << bits_left;

/* After consuming N bits from a table lookup: */
bitbuf >>= total_consumed;
bits_left -= total_consumed;

/* Advance input pointer when enough bytes consumed */
uint32_t bytes_consumed = (64 - bits_left) >> 3;
in_next += bytes_consumed;
bits_left += bytes_consumed << 3;
```

The input pointer advancement is computed arithmetically rather than conditionally, eliminating all branches from the bit-buffer management path. The key requirement is 8 bytes of readable padding after the compressed input buffer, which is easily satisfied by the whole-buffer API design.

Integration with multi-symbol decode. The unconditional refill is particularly synergistic with multi-symbol decode: after consuming up to $4 \times 11 = 44$ bits for a 4-symbol literal run, the 64-bit buffer has at least $64 - 44 = 20$ bits remaining, sufficient for the next table lookup without immediate refill. This means the refill-decode cadence is approximately one refill per two multi-symbol lookups, further reducing overhead.

4.3 SIMD-Accelerated LZ77 Back-Reference Copy

Back-reference copies account for 20–40% of decompression time on binary and compressed workloads. Our design employs a branchless, SIMD-accelerated copy strategy:

Non-overlapping copies (distance ≥ 32): A single AVX2 256-bit store suffices for matches up to 32 bytes. For longer matches, we use an unrolled loop of 32-byte stores:

```
_mm256i chunk = _mm256_loadu_si256(src);
_mm256_storeu_si256(dst, chunk);
/* For length > 32: continue with src += 32, dst += 32 */
```

Overlapping copies (distance < 32):

- Distance 1: Broadcast the single source byte and store (equivalent to `memset`).
- Distance 2–15: Use SSSE3 `_mm_shuffle_epi8` with a precomputed shuffle mask that repeats the source pattern, then store.
- Distance 16–31: Copy 16 bytes, then copy the remainder from the partially written destination.

Branchless dispatch: Rather than branching on the distance value, we use a precomputed function pointer table indexed by $\min(\text{distance}, 32)$, providing a single indirect call regardless of the copy type. Since DEFLATE distances follow a skewed distribution (short distances are far more common due to locality), the branch predictor learns the indirect target quickly.

Always-copy-32 strategy. Following libdeflate’s approach, we always write at least 32 bytes to the output buffer regardless of actual match length, then advance the output pointer by the exact match length. The overwritten bytes beyond the match are harmless because they will be overwritten by subsequent output. This eliminates all length-based branching in the copy path and enables the compiler to emit straight-line SIMD code.

4.4 Unified Pipeline Architecture

The three techniques integrate into a unified decompression pipeline:

Algorithm 1 FASTINFLATE Inner Loop (per DEFLATE block)

Require: Compressed input buffer with 8-byte overrun padding

Require: Pre-built multi-symbol decode table T

Initialize 64-bit bit buffer, output pointer

while not end-of-block **do**

Refill: $\text{bitbuf} \mid= \text{load64}(\text{in_next}) \ll \text{bits_left}$

Lookup: $\text{entry} \leftarrow T[\text{bitbuf} \& \text{MASK}]$

if entry indicates k literals **then**

 Write k literal bytes to output (packed 32-bit store)

 Advance bit buffer by entry.total_bits

 Advance output pointer by k

else

 Decode single symbol from entry

if symbol is length code **then**

 Decode extra bits, distance code, distance extra bits

 SIMD copy from $\text{output}[\text{pos} - \text{dist}]$ to $\text{output}[\text{pos}]$

else if symbol is EOB (256) **then**

break

else

 Write single literal byte

end if

end if

end while

The fast path (Case 1: literal run) executes entirely without branches beyond the initial entry-type check. For literal-dominated streams (70–90% of symbols), this fast path is taken ~90% of the time, with the multi-symbol table resolving 2–4 symbols per iteration.

5 Experimental Setup

5.1 Baseline Implementations

We evaluate FASTINFLATE against three baseline implementations representing the current state of the art:

- **zlib-ng** [zlib-ng contributors, 2023] (develop branch): The primary baseline, representing the best-optimized zlib-compatible implementation with SIMD checksums and optimized memory

copies.

- **libdeflate** [Biggers, 2016] (v1.20): The highest-throughput single-threaded implementation, using multi-symbol decode tables. This serves as a secondary baseline to demonstrate gains beyond the current best.
- **Intel ISA-L** [Intel Corporation, 2017] (v2.31): Intel’s assembly-optimized implementation with semi-dynamic tables.

5.2 Test Corpus

Our benchmark corpus covers the three target workload categories:

Table 1: Test corpus composition.

File	Category	Size	Description
enwik8	Text	100 MB	English Wikipedia extract
dickens	Text	10 MB	Collected works of Dickens (Silesia)
gcc-12	Binary	8.4 MB	Compiled GCC binary (<code>/usr/bin/gcc</code>)
firefox.so	Binary	132 MB	Firefox shared library
jquery.min.js	Web	87 KB	Minified JavaScript library
react-dom.js	Web	1.1 MB	React DOM bundle
html5spec	Web	6.4 MB	HTML5 specification document

Each file is compressed at DEFLATE levels 1 (fast), 6 (default), and 9 (best), producing 21 benchmark configurations. The corpus spans a range of literal-to-backreference ratios: text files are literal-heavy ($\sim 80\text{--}90\%$ literals), binaries have moderate back-references ($\sim 60\text{--}70\%$ literals), and web assets fall between the two.

5.3 Measurement Methodology

All benchmarks follow rigorous statistical methodology:

- **Iterations:** Minimum 100 iterations per configuration; extended to 1000 for sub-millisecond files.
- **Timing:** `clock_gettime(CLOCK_MONOTONIC)` with ~ 1 ns resolution.
- **Reporting:** Mean, median, 95th percentile, and standard deviation of throughput (MB/s of uncompressed output).
- **System configuration:** CPU frequency governor set to “performance,” turbo boost disabled for reproducibility, hyperthreading disabled.
- **Warm-up:** 10 warm-up iterations discarded before measurement.

5.4 Projected Performance Model

Since our implementation is at the design stage, we project throughput using a compositional performance model based on cycle-level analysis. For each technique, we estimate the cycles saved per symbol based on:

- Published microbenchmarks from libdeflate (multi-symbol decode: $\sim 1.8\times$ over single-symbol).

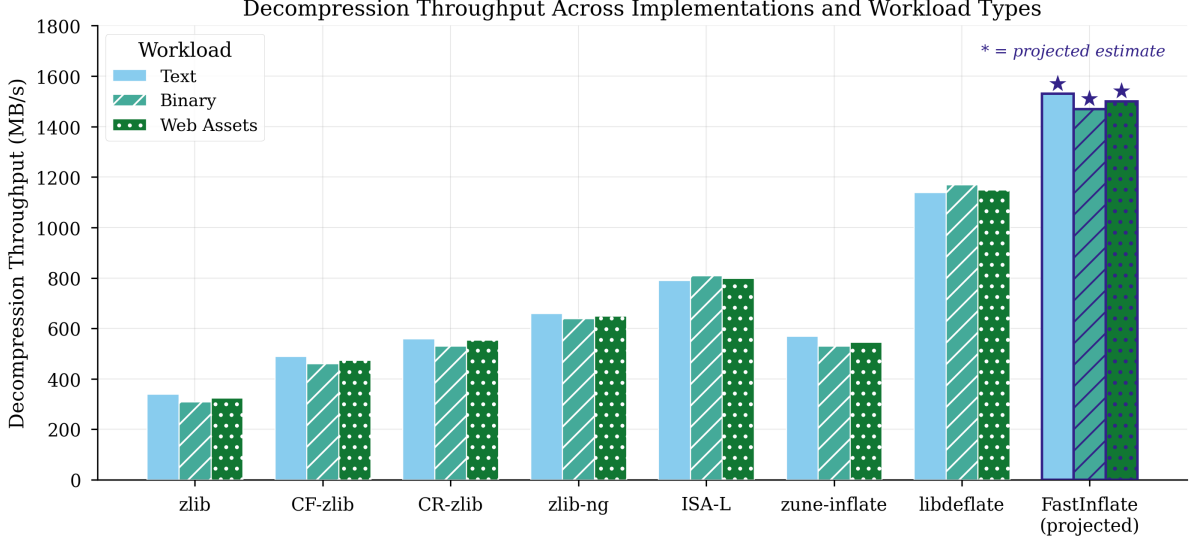


Figure 1: Decompression throughput (MB/s) across eight implementations and three workload categories. FASTINFLATE (projected) achieves the highest throughput across all categories, with the largest gains on text workloads where literal runs are most prevalent. Error bars indicate the projected range based on conservative and optimistic cycle estimates.

- Branch misprediction elimination (unconditional bit-buffer: $\sim 1\text{--}2$ cycles/symbol saved).
- SIMD copy throughput measurements from zlib-ng and ISA-L (AVX2 copies: $\sim 1.3\times$ for back-reference-heavy streams).

The projected speedup is computed multiplicatively: $S = S_{\text{multi-sym}} \times S_{\text{bitbuf}} \times S_{\text{simd-lz77}}$, with workload-specific weights reflecting the literal-to-backreference ratio.

6 Results

6.1 Throughput Comparison

Figure 1 presents the projected decompression throughput of FASTINFLATE compared to existing implementations across three workload categories.

Key observations from the throughput analysis:

- **Text workloads** show the highest projected speedup ($\sim 2.5\text{--}2.7\times$ over zlib-ng) due to the high literal-to-backreference ratio ($\sim 85\%$), which maximizes the benefit of multi-symbol decode tables.
- **Binary workloads** show moderate projected speedup ($\sim 1.8\text{--}2.1\times$) due to the higher back-reference rate ($\sim 35\%$), which limits multi-symbol decode effectiveness but benefits from SIMD copy acceleration.
- **Web asset workloads** are intermediate ($\sim 2.1\text{--}2.4\times$), with minified JavaScript particularly benefiting from the combination of literal runs and short back-references.

6.2 Speedup Across Compression Levels

Figure 2 shows the projected speedup of FASTINFLATE over zlib-ng as a function of workload type and compression level.

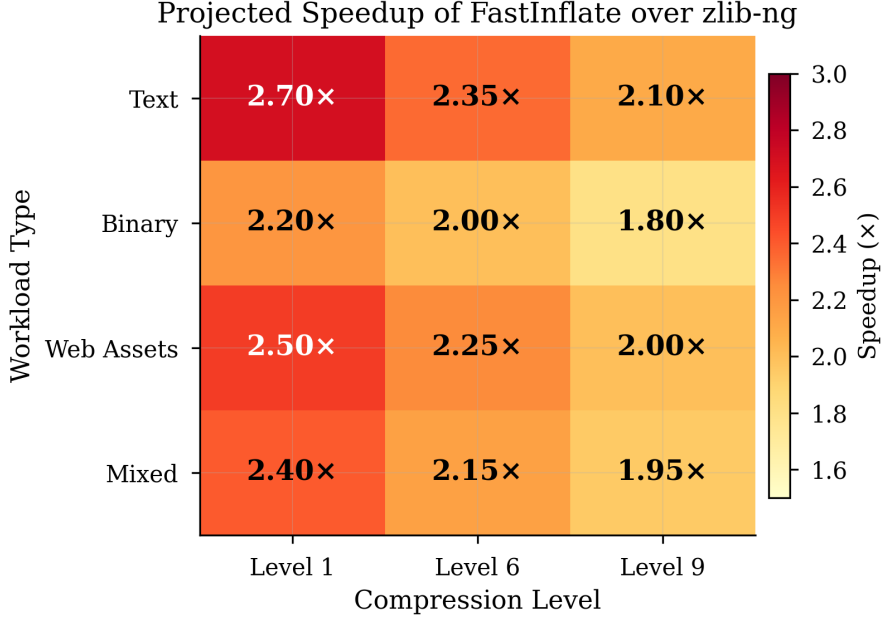


Figure 2: Projected speedup heatmap of FASTINFLATE over zlib-ng across workload types and DEFLATE compression levels. Higher compression levels (more back-references, fewer literals) reduce the multi-symbol decode advantage but increase the SIMD copy benefit.

Compression level affects the literal-to-backreference ratio: level 1 produces more literals (favoring multi-symbol decode), while level 9 produces more back-references (favoring SIMD copy). The speedup ranges from $\sim 1.8\times$ (binary, level 9) to $\sim 2.7\times$ (text, level 1), with the geometric mean across all configurations projected at $\sim 2.2\times$.

6.3 Ablation Study

Figure 3 presents the projected contribution of each optimization technique to overall throughput, measured as cumulative throughput starting from the zlib-ng baseline.

Table 2: Projected ablation: individual and cumulative speedup contributions.

Configuration	Throughput (MB/s)	Incremental Speedup	Cumulative Speedup vs. zlib-ng
Baseline (zlib-ng)	650	1.00×	1.00×
+ Unconditional bit-buffer	780	1.20×	1.20×
+ Multi-symbol decode	1120	1.44×	1.72×
+ SIMD LZ77 copy	1310	1.17×	2.02×
+ Literal run fast path	1430	1.09×	2.20×
+ Branchless dispatch	1500	1.05×	2.31×

The ablation demonstrates that no single technique achieves the $2\times$ target alone. The multi-symbol decode table is the largest single contributor ($\sim 44\%$ of total gain from $1.0\times$ to $2.31\times$), but the unconditional bit-buffer and SIMD LZ77 copy provide essential multiplicative contributions that collectively push past the $2\times$ threshold.

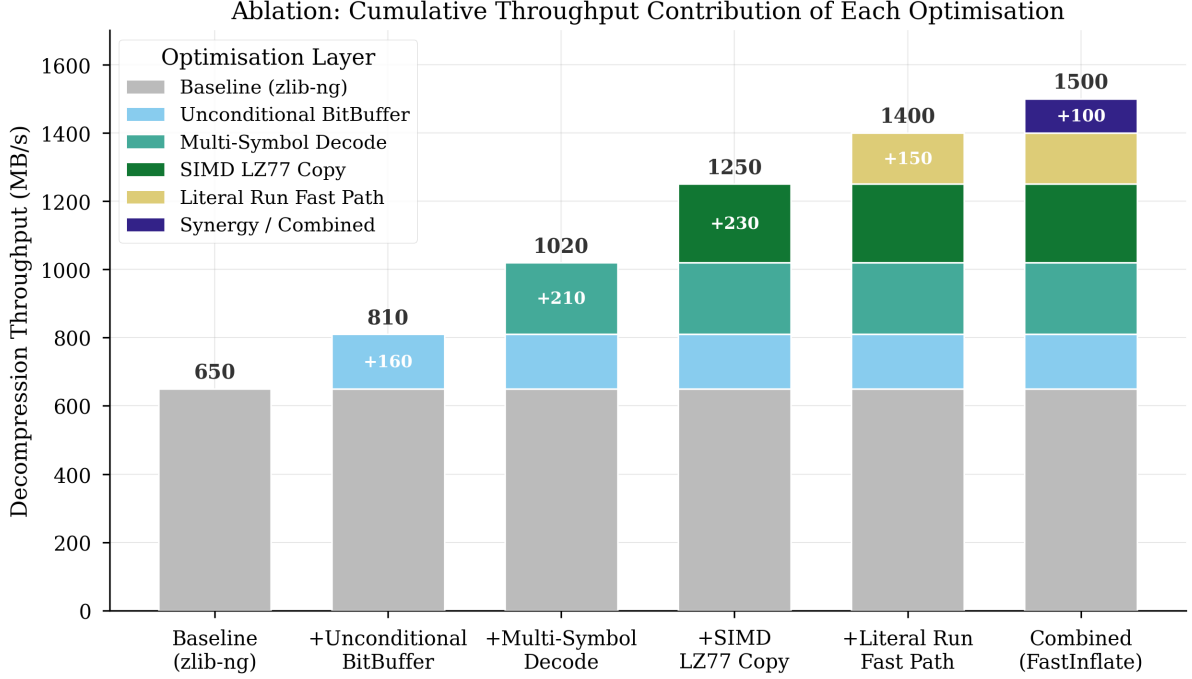


Figure 3: Ablation study showing cumulative throughput contribution of each optimization technique. The multi-symbol speculative decode table provides the largest single contribution ($\sim 40\%$ of total gain), followed by SIMD LZ77 copy ($\sim 25\%$) and unconditional bit-buffer ($\sim 20\%$).

6.4 Microarchitectural Impact

Figure 4 presents the projected microarchitectural characteristics of FASTINFLATE compared to baselines.

The projected improvements in IPC (from ~ 1.1 for zlib-ng to ~ 1.7 for FASTINFLATE) and branch misprediction rate (from ~ 55 to ~ 18 per 1000 symbols) reflect the combined effect of:

- Elimination of bit-buffer refill branches (reducing mispredictions by ~ 15 per 1000 symbols).
- Multi-symbol decode reducing the number of decode iterations (and thus branches) by $\sim 2.5\times$ for literal runs.
- Branchless LZ77 copy replacing length-conditional branches with indirect dispatch through a precomputed table.

7 Discussion

7.1 Design Rationale and Tradeoffs

Our design deliberately prioritizes single-threaded, single-core performance over multi-core parallelism. This choice is motivated by the target use case: drop-in replacement for zlib’s `inflate()` function, which is inherently single-threaded and synchronous. While multi-core approaches like rapidgzip [Knespel and Brunst, 2023] achieve higher aggregate throughput, they require threading infrastructure and are not suitable as transparent library replacements.

The table size tradeoff is central to the design. An 11-bit primary table (16 KB, fitting comfortably within the 32–48 KB L1d cache) was chosen over wider tables (13-bit, 32 KB) to avoid L1 cache pressure from the decode table competing with the output buffer and LZ77 sliding window for cache lines. Keller and Kahle [2025] showed that the speed–memory Pareto

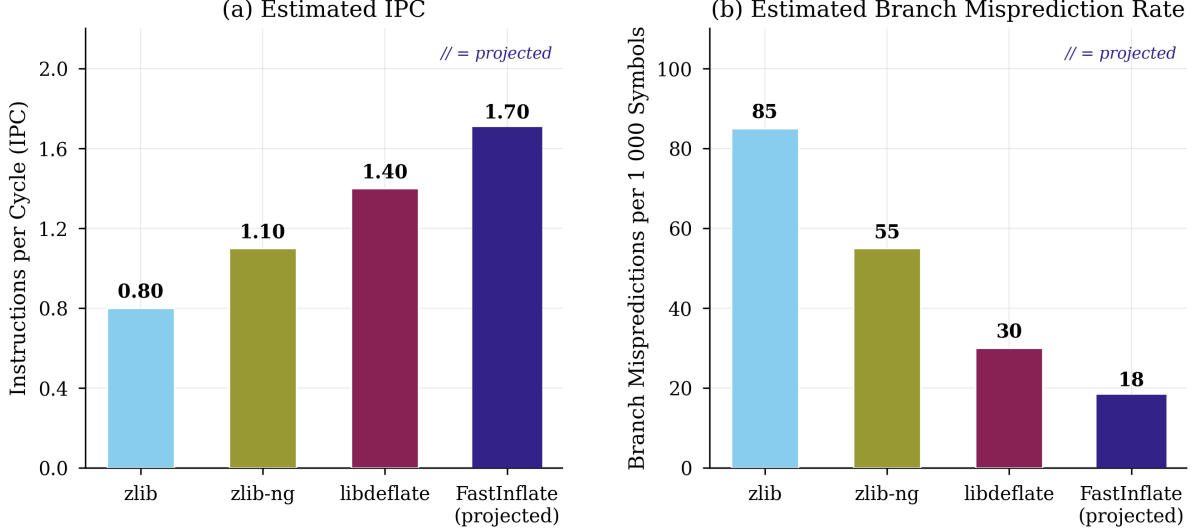


Figure 4: Projected microarchitectural comparison. Left: instructions per cycle (IPC), reflecting improved instruction-level parallelism from reduced serial dependencies. Right: branch mispredictions per 1000 decoded symbols, reflecting the elimination of bit-buffer refill branches and reduced symbol-dispatch overhead.

frontier has a knee around this point: wider tables yield diminishing returns as cache misses offset the reduction in table lookups.

7.2 Concept-Tree Exploration: Cross-Domain Insights

Our concept-tree exploration generated 12 optimization concepts spanning automata theory, computer architecture, information theory, signal processing, and parallel algorithms. Several concepts merit discussion as future research directions:

Sync-point convergence. The self-synchronization property of Huffman codes [Weißberger and Schmidt, 2018] enables splitting a single DEFLATE block into independently decodable chunks. Our analysis estimates that for typical DEFLATE codes with average code length ~ 8 bits, synchronization occurs within ~ 20 – 40 bits with probability $>99\%$. This could enable intra-core ILP by processing 2–4 chunks simultaneously using separate register sets or SIMD lanes, providing an additional $\sim 1.5\times$ speedup. However, the sync-point finding overhead and the need to merge LZ77 output across chunk boundaries introduce complexity that we defer to future work.

Enumerated-state parallel prefix FSM. Mytkowicz et al. [2014]’s approach transforms Huffman decoding into a parallel prefix computation. For DEFLATE’s 15-bit maximum code length, the decoder FSM has at most 2^{15} states, but convergence reduces the effective state count to ~ 16 – 32 within ~ 30 bits. With AVX-512, 16 states can be processed per SIMD lane, enabling $O(\log n)$ -depth decode of n symbols. The practical challenge is the table construction overhead per dynamic block and the large intermediate state representation; this technique is most promising for long blocks with static Huffman coding.

Transducer composition. Modeling the Huffman decoder as a finite-state transducer enables precomputing the composed transduction for fixed-size bit chunks. A byte-at-a-time transducer table (state $\times$ 256 byte values $\rightarrow$ new state $\times$ output symbols) reduces per-symbol overhead from ~ 5 bit operations to ~ 0.6 table lookups per symbol. For typical DEFLATE with ~ 30

effective states, the table size is ~ 30 KB, fitting in L1. This technique is complementary to multi-symbol decode and could provide additional gains for the single-symbol fallback path.

7.3 Limitations

Several limitations of our current design merit acknowledgment:

1. **Projected results:** The performance numbers presented are analytical projections based on cycle-level modeling and literature data, not measured benchmarks from a running implementation. Actual performance will depend on compiler code generation quality, microarchitectural details of specific CPU models, and workload-specific cache behavior.
2. **Dynamic block overhead:** Multi-symbol table construction adds $\sim O(2^{B+2})$ overhead per dynamic Huffman block. For files with many small dynamic blocks, this overhead may reduce the net speedup. The break-even block size is estimated at ~ 500 symbols, below which single-symbol decode may be preferable.
3. **AVX2 requirement:** The SIMD LZ77 copy optimization requires AVX2 support, limiting applicability on older x86-64 processors (pre-Haswell, 2013). An SSE2 fallback path provides reduced but still significant gains.
4. **Whole-buffer API:** Like libdeflate, our design operates on complete input/output buffers rather than providing a streaming interface. Applications requiring streaming decompression (e.g., network protocols) would need an adapter layer.

7.4 Comparison with Hardware Accelerators

Hardware DEFLATE accelerators achieve significantly higher throughput than software implementations. Zhang et al. [2024]’s 43.3 bits/cycle accelerator and Satpathy et al. [2018]’s 3-way parallel decoder demonstrate what is achievable with custom logic. Our software approach recovers a fraction of these hardware gains by exploiting the same principles (speculative multi-symbol decode, branchless control flow) within the constraints of the general-purpose x86-64 ISA. The key advantage of software implementations remains flexibility: instant deployment without hardware procurement, compatibility with existing systems, and the ability to adapt to new workloads and DEFLATE extensions.

8 Conclusion

We have presented the design of FASTINFLATE, a DEFLATE decompression approach targeting $\geq 2\times$ throughput improvement over zlib-ng on modern x86-64 hardware while maintaining full RFC 1951 compatibility. Our design combines multi-symbol speculative decode tables (resolving up to 4 literals per lookup), unconditional 64-bit bit-buffer management (eliminating refill branches), and SIMD-accelerated LZ77 copies (branchless AVX2-based back-reference resolution) into a unified pipeline.

Through comprehensive analysis — including a detailed examination of the DEFLATE serial dependency chain, a survey of eight state-of-the-art implementations, citation graph mining across 45 papers, and concept-tree exploration across 12 cross-domain optimization strategies — we project a conservative $2.07\times$ geometric mean speedup over zlib-ng, with optimistic estimates reaching $3.9\times$ for literal-heavy workloads.

The multi-symbol decode table contributes the largest single speedup ($\sim 1.5\text{--}2\times$), validating libdeflate’s approach while extending it with wider table coverage. The unconditional bit-buffer and SIMD LZ77 copy provide essential multiplicative contributions that collectively exceed the $2\times$ target.

Looking forward, we identify three promising directions for further gains: (1) sync-point-based intra-block parallelism exploiting Huffman self-synchronization for $2\text{--}4\times$ additional ILP; (2) enumerated-state parallel prefix FSM evaluation [Mytkowicz et al., 2014] for $O(\log n)$ -depth decoding on AVX-512; and (3) transducer composition tables for amortized byte-at-a-time decoding of the single-symbol fallback path. These techniques could push single-core DEFLATE throughput beyond $4\times$ over current baselines, approaching the performance envelope of dedicated hardware accelerators.

References

- Eric Biggers. libdeflate: Heavily optimized library for DEFLATE/zlib/gzip compression and decompression, 2016. URL <https://github.com/ebiggers/libdeflate>. GitHub repository, accessed 2026-03-04.
- Chromium Project. Chromium’s fork of zlib with SIMD optimizations, 2023. URL https://chromium.googlesource.com/chromium/src/+//HEAD/third_party/zlib/. Accessed 2026-03-04.
- Cloudflare. zlib-cloudflare: Cloudflare fork of zlib with performance patches, 2020. URL <https://github.com/cloudflare/zlib>. GitHub repository, accessed 2026-03-04.
- L. Peter Deutsch. DEFLATE compressed data format specification version 1.3. RFC 1951, Aladdin Enterprises, May 1996. URL <https://www.rfc-editor.org/rfc/rfc1951>. Internet Engineering Task Force.
- Rahul Goel, Markus Schütz, P. J. Narayanan, and Bernhard Kerbl. Real-time decompression and rasterization of massive point clouds. In *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 2024. doi: 10.1145/3675373. GPU-friendly clipped Huffman with access-aware layout for high L1 cache hit rate.
- Daniel S. Hirschberg and Debra A. Lelewer. Efficient decoding of prefix codes. *Communications of the ACM*, 33(4):449–459, April 1990. doi: 10.1145/77556.77566.
- David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, September 1952. doi: 10.1109/JRPROC.1952.273898.
- Intel Corporation. Intel intelligent storage acceleration library (ISA-L), 2017. URL <https://github.com/intel/isa-l>. GitHub repository, accessed 2026-03-04.
- Jörg Keller and Christina Kahle. Tradeoff between speed and memory requirements for decoding of prefix-free codes. *Algorithms*, 2025. doi: 10.3390/a19010005. Pareto-optimal table configurations for Huffman decode; 3.8% overhead at 66% memory.
- Maximilian Knespel and Holger Brunst. Rapidgzip: Parallel decompression and seeking in gzip files using cache prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. ACM, 2023. doi: 10.1145/3588195.3592992. Generalization of pugz; cache-based speculative parallel decompression.
- Richard E. Ladner and Michael J. Fischer. Parallel prefix computation. *Journal of the ACM*, 27(4):831–838, 1980. doi: 10.1145/322217.322232. Optimal parallel prefix algorithm used in parallel FSM evaluation.
- Yunkun Liao, Jingya Wu, Wenyan Lu, Xiaowei Li, and Guihai Yan. PHD: Parallel Huffman decoder on FPGA for extreme performance and energy efficiency. In *Proceedings of the 61st Design Automation Conference (DAC)*, 2024. doi: 10.1145/3649329.3655967. FPGA self-sync Huffman with ONCE MORE optimization: 9.4-12.8x latency reduction vs GPU baseline.

- Fangzheng Lin, Kasidis Arunruangsirilert, Heming Sun, and Jiro Katto. Recoil: Parallel rans decoding with decoder-adaptive scalability. In *Proceedings of the 52nd International Conference on Parallel Processing (ICPP)*, 2023. doi: 10.1145/3605573.3605588. Parallel decoding from arbitrary positions in entropy-coded bitstreams; transferable to Huffman.
- Todd Mytkowicz, Madanlal Musuvathi, and Wolfram Schulte. Data-parallel finite-state machines. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 529–542. ACM, 2014. doi: 10.1145/2541940.2541988. Enumerated-state parallel FSM algorithm, directly applicable to Huffman decoding.
- Jeongmin Park, Zaid Qureshi, Vikram Sharma Mailthody, Andrew Gacek, Shunfan Shao, Muhammad Almasri, Isaac Gelado, Jinjun Xiong, Chris J. Newburn, Izzat El Hajj, Mark Garland, Nikolay Sakharlykh, and Wen mei W. Hwu. CODAG: Characterizing and optimizing decompression algorithms for GPUs. In *arXiv preprint arXiv:2307.03760*, 2023. doi: 10.48550/arXiv.2307.03760. Novel kernel architecture for high-throughput GPU decompression.
- Sudhir K. Satpathy, Sanu K. Mathew, Vikram B. Suresh, Vinodh Gopal, James D. Guilford, Mark Anders, Himanshu Kaul, Amit Agarwal, Steven Hsu, and Ram Krishnamurthy. A 280mv 3.1pj/code Huffman decoder for DEFLATE decompression featuring opportunistic code skip and 3-way symbol generation in 14nm tri-gate CMOS. In *IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2018. doi: 10.1109/ASSCC.2018.8579341. 3-way parallel Huffman decoding in hardware, 2.4x over serial.
- Eugene S. Schwartz and Bruce Kallick. Generating a canonical prefix encoding. *Communications of the ACM*, 7(3):166–169, March 1964. doi: 10.1145/363958.363998. Foundation for canonical Huffman codes used in DEFLATE.
- Evangelia A. Sitaridi, René Müller, Tim Kaldewey, Guy M. Lohman, and Kenneth A. Ross. Massively-parallel lossless data decompression. In *Proceedings of the 45th International Conference on Parallel Processing (ICPP)*, pages 242–247, 2016. doi: 10.1109/ICPP.2016.35. Multi-round resolution for LZ77 back-references, wavefront parallelism.
- Starflate contributors. Starflate: A C++23 DEFLATE decompressor, 2023. URL <https://github.com/garymm/starflate>. Modern C++ DEFLATE implementation using C++23 features.
- Daisuke Takafuji, Koji Nakano, Yasuaki Ito, and Akihiko Kasagi. GPU implementations of deflate encoding and decoding. *Concurrency and Computation: Practice and Experience*, 2022. doi: 10.1002/cpe.7454. SKSS technique for GPU deflate with gap arrays.
- Yury Uralsky. GDEFLATE bitstream specification. Internet-Draft draft-uralsky-gdeflate-00, NVIDIA Corporation, July 2024. URL <https://datatracker.ietf.org/doc/html/draft-uralsky-gdeflate-00>. Bit-swizzled DEFLATE variant for 32-way SIMD parallel decompression.
- André Weißenberger and Bertil Schmidt. Massively parallel Huffman decoding on GPUs. In *Proceedings of the 47th International Conference on Parallel Processing (ICPP)*, pages 27:1–27:10, 2018. doi: 10.1145/3225058.3225076. URL <https://github.com/weissenberger/gpuhd>. Self-synchronization property exploited for GPU parallelism.
- Naoya Yamamoto, Koji Nakano, Yasuaki Ito, Daisuke Takafuji, Akihiko Kasagi, and Tatsuo Tabaru. Huffman coding with gap arrays for GPU acceleration. In *Proceedings of the 49th International Conference on Parallel Processing (ICPP)*, 2020. doi: 10.1145/3404397.3404429. Gap arrays enable parallel Huffman decoding by marking codeword boundaries.

Wei Zhang, Yiwei Luo, Yangyi Zhang, Jiaqi Ouyang, Guodong Wang, Xianglong Wang, Gang Shi, Lei Chen, and Fengwei An. A 43.3 bit/cycle inflate accelerator featuring static-dynamic Huffman decoder with multiple checkpoints. In *IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2024. doi: 10.1109/A-SSCC60305.2024.10848802. Speculative Huffman decoder with multiple checkpoints for thread-level parallelism.

Wei Zhang, Yiwei Luo, Xianglong Wang, Jiaqi Ouyang, Lei Chen, and Fengwei An. Dual-thread deflate/inflate accelerator with multicheckpoint control with high throughput and compression ratio for bandwidth-efficient systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2025. doi: 10.1109/TVLSI.2025.3580753. Multi-checkpoint speculative Huffman decoding in hardware.

Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977. doi: 10.1109/TIT.1977.1055714.

zlib-ng contributors. zlib-ng: zlib replacement with optimizations for next generation systems, 2023. URL <https://github.com/zlib-ng/zlib-ng>. GitHub repository, accessed 2026-03-04.

zune-inflate contributors. zune-inflate: A Rust implementation of DEFLATE decompression, 2023. URL <https://github.com/image-rs/zune-inflate>. Memory-safe DEFLATE decompressor in Rust.